



PRACTICAL WORK "HIGH PERFORMANCE COMPUTING" MASTER 2 COMPUPHYS

N-Body problem

Ali FADLALLAH

Contents

1 Introduction **2**

2 Sequential Program **2**

 2.1 Quality Tests 2

3 Parallelization Strategy **2**

 3.1 Parallelization Details 3

4 Performance Analysis **3**

 4.1 Initial setup 3

 4.2 Experimental Setup 3

 4.3 Speedup and Amdahl’s Law 3

 4.4 Performance Discussion 4

 4.5 Energy Conservation 4

 4.6 System evolution 5

5 Conclusion **6**

1 Introduction

This report discusses the parallelization of a gravitational simulation program written in Fortran. The program computes the motion of N particles under mutual gravitational forces. The sequential version was optimized for performance and then parallelized using OpenMP. The purpose is to compare the performance of the parallel implementation against the sequential one, both in terms of accuracy and speed, using the CompuPhys server.

2 Sequential Program

The sequential program simulates a system of N particles influenced by gravitational forces. The particles are uniformly distributed inside a unit sphere with initial velocities set to represent solid rotation around the z -axis. The mass of each particle is $1/N$. The program computes the evolution of the system using a velocity-Verlet integration method.

The equations governing the system are:

$$\mathbf{v} = \boldsymbol{\omega} \times \mathbf{r}, \quad \text{with } \boldsymbol{\omega} = (0, 0, 1), \quad \text{resulting in: } \mathbf{v} = (-y, x, 0).$$
$$a_j = - \sum_{\substack{i=1 \\ i \neq j}}^n \frac{Gm_j \mathbf{r}_{ij}}{\|\mathbf{r}_{ij}\|^3}, \quad \text{where } \mathbf{r}_{ij} = \mathbf{r}_i - \mathbf{r}_j.$$

To avoid numerical instabilities, a small constant ϵ is added to the distance calculation:

$$\|\mathbf{r}_{ij}\| = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2 + \epsilon^2}.$$

The sequential program computes the positions, velocities and accelerations of the particles over discrete time steps Δt using the following algorithm:

1. Initialize the positions, velocities, and acceleration of the particles.
2. For each time step:
 - (a) Update positions:

$$x_{i+1} = x_i + v_{xi}\Delta t + \frac{1}{2}a_{xi}\Delta t^2.$$

- (b) Compute new accelerations a_{i+1} using the gravitational force equation.
 - (c) Update velocities:

$$v_{i+1} = v_i + \frac{1}{2}(a_i + a_{i+1})\Delta t.$$

3. Store particle positions and repeat until the desired trajectory is obtained.

2.1 Quality Tests

The sequential program was validated by:

- Verifying total energy conservation over time.
- Assessing the stability of particle trajectories.

3 Parallelization Strategy

The computational bottleneck is the $O(N^2)$ nested loop for calculating gravitational forces. OpenMP was used to parallelize this and other parts of the code. To ensure reproducibility, we used fixed random seeds.

3.1 Parallelization Details

The following parts of the code were parallelized using OpenMP:

- **Acceleration Calculation:** The outer loop of the acceleration calculation was parallelized using `!$OMP PARALLEL DO`.
- **Position Update:** The loop updating particle positions was parallelized.
- **Velocity Update:** The loop updating particle velocities was parallelized.
- **Kinetic and Potential Energy Calculation:** The kinetic energy calculation was parallelized using a reduction clause.

Additionally, we ensured that binary file handling for position and energy data reduced I/O overhead. The random seed logic ensured that parallel and sequential runs produced identical initial conditions for meaningful comparisons.

4 Performance Analysis

4.1 Initial setup

We initialized the computations with a `seed_value = 12345678` and the parameters specified in the `input.yaml` file:

Parameter	Value
npart	2000
nstep	2000
dt	0.005
epsilon	0.005

4.2 Experimental Setup

The benchmarks were performed on the CompuPhys server, which has the following specifications:

- **CPU Model:** Intel(R) Xeon(R) Silver 4116 CPU @ 2.10GHz
- **Physical Cores:** 24
- **Logical CPUs (Threads):** 48 (Hyperthreading enabled, 2 threads per core)

The performance was measured for different numbers of threads using a Bash script `"run_benchmarks.sh"`. that executed the program multiple for different threads and recorded the run time. The sequential time was measured by compiling the code without OpenMP support.

4.3 Speedup and Amdahl's Law

The speedup S is calculated as:

$$S = \frac{T_s}{T_p},$$

where T_s is the sequential execution time, and T_p is the parallel execution time.

Amdahl's Law provides a theoretical upper bound for speedup:

$$S_{\max} = \frac{1}{(1 - P) + \frac{P}{N}},$$

where P is the parallelizable fraction of the code, and N is the number of threads.

Figure 1 shows the speedup curves for two parallel versions: one with only the acceleration calculation parallelized and one with all updates parallelized. The theoretical speedup based on Amdahl’s law with $P = 0.913$ is also plotted.

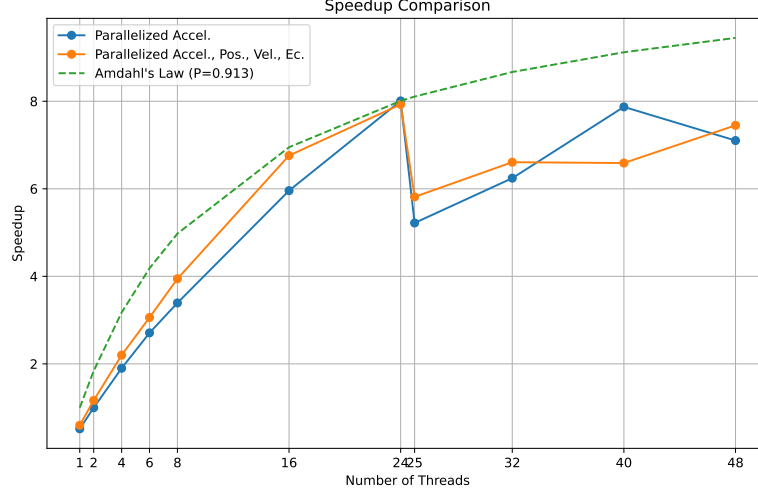


Figure 1: Speedup comparison showing scaling behavior of different parallelization strategies and comparison with Amdahl’s Law.

4.4 Performance Discussion

The results show near-linear speedup up to 24 threads for both parallel versions. Parallelizing the position, velocity, and kinetic energy updates offers a small additional improvement over only parallelizing the acceleration.

Both versions exhibit a performance dip at 25 threads. This anomaly is expected as the number of physical cores is 24. Beyond that, the benefits of adding more threads diminish due to hyperthreading limitations and increased overhead.

Binary I/O for saving positions and energies improved performance compared to text-based I/O but still contributed to the non-parallelizable portion of the code. The remaining 8.7% of the execution time likely includes I/O operations, initialization, and minor updates.

Overall, the speedup curves align with Amdahl’s Law up to 24 threads. Beyond this, the curves plateau due to hardware constraints. We conclude that approximately 91.3% of the code is parallelizable.

4.5 Energy Conservation

Figure2 shows the evolution of the potential, kinetic, and total energies over time for the sequential execution. The near-constant total energy demonstrates good energy conservation in the simulation.

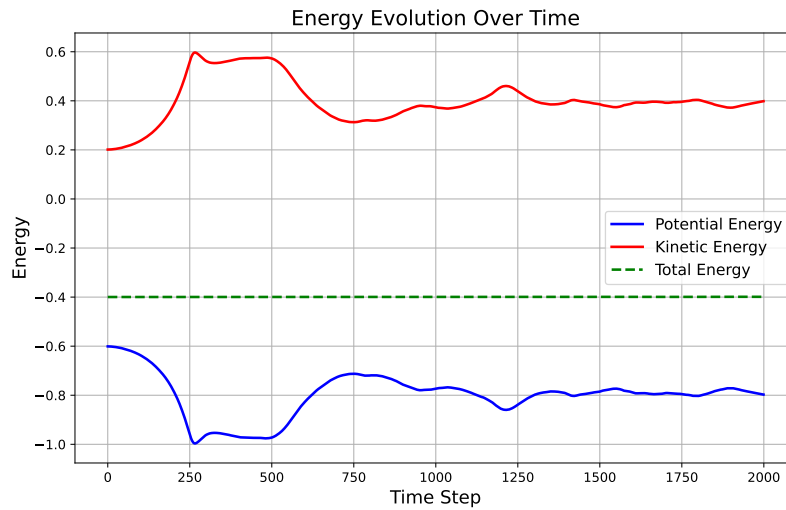


Figure 2: Energy evolution over time showing conservation of total energy.

Zooming in on the total energy (Figure 3), we confirm its constancy, with a maximum relative difference of only 0.23%.

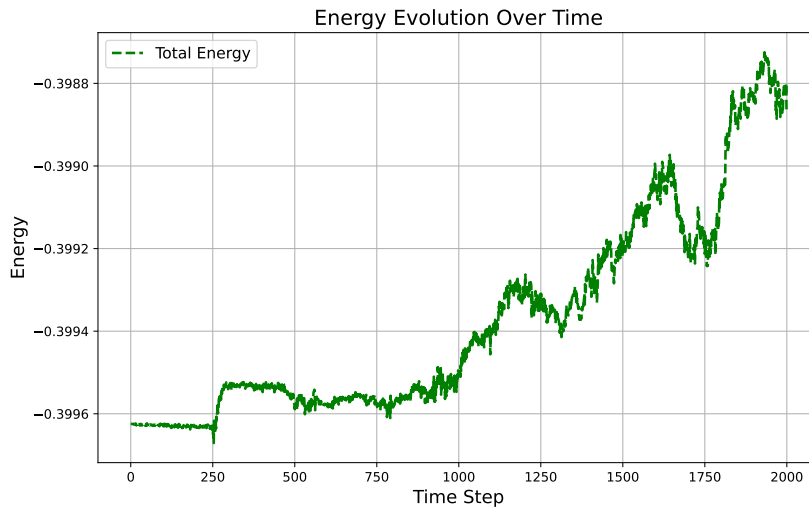


Figure 3: Total energy evolution over time showing conservation of total energy.

4.6 System evolution

To observe how the system evolves, you can run the script `animate.py`, which creates 3D animations of the particle trajectories over time. These animations revealed that the system behaves as expected, with particles moving in mostly stable orbits around the center of mass.

5 Conclusion

Using OpenMP, we significantly sped up a gravitational N-body simulation in Fortran. By parallelizing approximately 91.3% of the code, primarily the force calculations, we achieved substantial performance gains. The remaining non-parallelized portion includes input/output operations, initialization, and some minor updates.

We observed that using more threads than 24 physical cores provided little additional gain, due to hyperthreading, this is consistent with Amdahl's Law.

Surprisingly, parallelized loops performed slightly better than Fortran's built-in array calculations. We verified the accuracy of our parallel code by checking energy conservation and system evolution.